Slurm 入門(動かし方・使い方)

0. コンテナファイル取得 (enroot)

デフォルトのコンテナイメージを利用するための準備手順です。

- **デフォルトコンテナイメージ**: $HOME/nvidia+pytorch+25.11-py3.sqsh
- **取得コマンド**: enroot import docker://nvcr.io/nvidia/pytorch:25.11-py3
- **取得の確認**: ls -lh $HOME/nvidia+pytorch+25.11-py3.sqsh

1. 最初に確認すること

ジョブを実行する前に、以下のコマンドでシステムの状態を確認します。

- パーティションの状態を確認する: sinfo
- ノードの状態を確認する: sinfo -N
- 自分のジョブの状態を確認する: squeue -u $USER

2. 対話実行 (srun)

デフォルトでコンテナイメージを指定して実行します。

CPU のみでシェルを取得する場合:

```
srun -p gpu -c 4 --mem=16G \
--container-image=$HOME/nvidia+pytorch+25.11-py3.sqsh \
--container-mounts=$HOME:$HOME --pty bash
```

GPU 1 枚でシェルを取得する場合:

```
srun -p gpu --gres=gpu:1 -c 8 --mem=32G \
--container-image=$HOME/nvidia+pytorch+25.11-py3.sqsh \
--container-mounts=$HOME:$HOME --pty bash
```

3. バッチ実行 (sbatch)

スクリプトファイルを作成し、バックグラウンドでジョブを投入します。

job.sh の記述例:

```
#!/bin/bash
#SBATCH -J sample-gpu
#SBATCH -p gpu
```

```
#SBATCH --gres=gpu:1
#SBATCH -c 8
#SBATCH --mem=32G
#SBATCH -o slurm-%j.out
#SBATCH --container-image=$HOME/nvidia+pytorch+25.11-py3.sqsh
#SBATCH --container-mounts=$HOME:$HOME
echo "JOB_ID=$SLURM_JOB_ID"
hostname
nvidia-smi
python train.py
```

ジョブの投入コマンド: sbatch job.sh

4. ジョブ監視・停止

投入したジョブの管理コマンドです。

- キューの確認: squeue -u $USER
- ジョブの詳細確認: scontrol show job <JOB_ID>
- ジョブの停止: scancel <JOB_ID>

5. よく使う指定オプション

オプション	説明
-c <num>	CPU コア数
--mem=<size>	メモリ (例: --mem=64G)
--gres=gpu:<num>	GPU 枚数
-p gpu	パーティション指定
-o slurm-%j.out	出力ファイル (%j は JobID に置換されます)
--container-image=...	コンテナイメージ
--container-mounts=...	ローカルディレクトリのマウント

--container-mounts の指定例

- 単一ディレクトリ: --container-mounts=$HOME:$HOME
- 複数ディレクトリ: --container-mounts=$HOME:$HOME,$HOME/datasets:/workspace/datasets

マウントを利用するメリット

- ホストのコード、データ、出力をコンテナから直接扱えます。
- コンテナを再作成することなく、ファイルの変更を即座に反映できます。

マウントを利用するデメリット

- 権限の不整合により、読み書きエラーが起こる可能性があります。
- ホストのディレクトリ構成に依存するため、移植性が下がります。

運用上の推奨事項

- 必要最小限のパスのみをマウントしてください。
- 基本は --container-mounts=$HOME:$HOME とし、必要な時のみ追加することを推奨します。

6. よくあるトラブルと確認ポイント

- **PD (Pending) 状態が続く場合**: squeue -j <JOB_ID> -o "%.18i %.10T %.30R" を実行して理由を確認してください。
- **GPU が見えない場合**: ジョブ内で nvidia-smi を実行し、--gres=gpu:<num> の指定が正しく行われているか確認してください。
- **実行できない・失敗する場合**: slurm-<JOB_ID>.out のログを確認してください。解決しない場合は管理者へ連絡してください (ユーザー追加や設定変更・障害対応は運営事務局が担当です)。

7. マルチノード実行 (分散学習 / MPI)

本クラスタはH100・A5000 GPUサーバーに各 1 つ設置されています。

ノード割当の確認

割り当てられたノード一覧と各ノードの状態は以下で確認できます。

```
# 割り当てられたノード一覧
scontrol show hostnames "$SLURM_JOB_NODELIST"
# 各ノードでの確認
srun -N2 -n2 bash -lc 'hostname; echo
"CUDA_VISIBLE_DEVICES=$CUDA_VISIBLE_DEVICES"; nvidia-smi -L | wc -l'
```

PyTorch DDP (torchrun) テンプレート (推奨)

1 ノードあたり 1 プロセス (torchrun) を起動し、各ノード内で GPU 枚数ぶんのワーカープロセスを立ち上げる推奨方式です。

ddp_torchrun.sh の記述例:

```
#!/bin/bash
#SBATCH -J ddp-torchrun
#SBATCH -p gpu
#SBATCH --nodes=2
#SBATCH --ntasks-per-node=1
#SBATCH --gres=gpu:8
#SBATCH -c 32
#SBATCH --mem=200G
#SBATCH -o slurm-%j.out
#SBATCH --container-image=$HOME/nvidia+pytorch+25.11-py3.sqsh
#SBATCH --container-mounts=$HOME:$HOME
set -euo pipefail
MASTER_ADDR=$(scontrol show hostnames "$SLURM_JOB_NODELIST" | head -n
1)
MASTER_PORT=$((10000 + SLURM_JOB_ID % 50000))
srun --kill-on-bad-exit=1 bash -lc '
torchrun \
--nnodes=$SLURM_JOB_NUM_NODES \
--nproc_per_node=8 \
--node_rank=$SLURM_NODEID \
--rdzv_id=$SLURM_JOB_ID \
--rdzv_backend=c10d \
--rdzv_endpoint=$MASTER_ADDR:$MASTER_PORT \
train.py
'
```

PyTorch DDP (srun 1GPU=1task) テンプレート

Slurm が 1 タスク = 1 GPU でプロセスを起動し、各タスクが RANK, WORLD_SIZE, MASTER_ADDR, MASTER_PORT を環境変数で受け取る方式です。

ddp_srun_task.sh の記述例:

```
#!/bin/bash
#SBATCH -J ddp-srun-task
#SBATCH -p gpu
#SBATCH --nodes=2
#SBATCH --ntasks-per-node=8
#SBATCH --gres=gpu:8
#SBATCH -c 8
#SBATCH --mem=200G
#SBATCH -o slurm-%j.out
#SBATCH --container-image=$HOME/nvidia+pytorch+25.11-py3.sqsh
#SBATCH --container-mounts=$HOME:$HOME
set -euo pipefail
MASTER_ADDR=$(scontrol show hostnames "$SLURM_JOB_NODELIST" | head -n
1)
MASTER_PORT=$((10000 + SLURM_JOB_ID % 50000))
export MASTER_ADDR MASTER_PORT
export WORLD_SIZE=$SLURM_NTASKS
```

```
srun --kill-on-bad-exit=1 bash -lc '
export RANK=$SLURM_PROCID
export LOCAL_RANK=0
python -u train.py
'
```

Python 側の最小の初期化イメージ:

```
import torch
import torch.distributed as dist
dist.init_process_group(backend="nccl", init_method="env://")
torch.cuda.set_device(0)
```

MPI テンプレート (参考)

MPI 実行方式は、MPI 実装や Slurm の設定に依存します。

まずは以下のコマンドで利用可能な MPI ランチャを確認してください。 srun --mpi=list

mpi_job.sh の記述例:

```
#!/bin/bash
#SBATCH -J mpi-sample
#SBATCH -p gpu
#SBATCH --nodes=2
#SBATCH --ntasks-per-node=8
#SBATCH -c 2
#SBATCH --mem=8G
#SBATCH -o slurm-%j.out
#SBATCH --container-image=$HOME/nvidia+pytorch+25.11-py3.sqsh
#SBATCH --container-mounts=$HOME:$HOME
set -euo pipefail
srun hostname
```